

JAVA — Révision

1.Syntaxe 2.POO 3.Collections 4.Streams 5.Design Patterns 6.JUnit & JDBC

1. Variables et types

Java est **typé statiquement** : chaque variable a un type fixé dès sa déclaration, et ce type ne change jamais.

```
int age = 21;           // entier
double prix = 19.99;    // décimal
boolean actif = true;   // vrai ou faux
char lettre = 'A';      // un seul caractère
String nom = "Ainhua";  // chaîne de caractères (pas une primitive)
```

Deux familles de types

- **Types primitifs** : int, double, boolean, char, long, float, byte, short — stockent directement la valeur en mémoire.
- **Types objets** (référence) : String, Integer, ArrayList, etc. — stockent une référence vers un objet en mémoire.

```
int x = 5;           // primitif : valeur directe
Integer y = 5;       // objet : "autoboxing" automatique vers Integer
```

À retenir : un primitif ne peut jamais être **null**, alors qu'un objet (comme Integer) le peut.

2. Structures de contrôle

Condition if / else

```
if (age >= 18) {
    System.out.println("Majeur");
} else {
    System.out.println("Mineur");
}
```

Boucle for classique

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Boucle while

```
int compteur = 0;
while (compteur < 3) {
    compteur++;
}
```

Switch moderne (Java 14+)

```
String jour = switch (3) {
    case 1, 7 -> "Week-end";
    case 2, 3, 4, 5, 6 -> "Semaine";
    default -> "Inconnu";
};
```

3. Méthodes

```
public class Calcul {
    public static int additionner(int a, int b) {
        return a + b;
    }
}
```

Mot-clé	Rôle
public	Visibilité : accessible depuis n'importe où
static	Appartient à la classe, pas à une instance
int	Type de retour de la méthode
(int a, int b)	Paramètres de la méthode

JAVA — Thème 2

Programmation Orientée Objet : classes, héritage, interfaces, polymorphisme

1. Classes et objets

Une **classe** est un plan de construction, un **objet** est une instance concrète créée à partir de ce plan.

```
public class Personne {  
    // attributs (état)  
    private String nom;  
    private int age;  
  
    // constructeur  
    public Personne(String nom, int age) {  
        this.nom = nom;  
        this.age = age;  
    }  
  
    // méthode (comportement)  
    public void direBonjour() {  
        System.out.println("Bonjour, je suis " + nom);  
    }  
}
```

```
Personne p1 = new Personne("Ainhua", 21); // création d'un objet  
p1.direBonjour();
```

- **this** désigne l'instance courante (distingue l'attribut du paramètre).
- **private** : l'attribut n'est accessible que depuis l'intérieur de la classe → on passe par des **getters/setters**.

```
public String getNom() {  
    return nom;  
}  
  
public void setAge(int age) {  
    this.age = age;  
}
```

2. Héritage

Une classe peut **hériter** d'une autre avec **extends** : elle récupère ses attributs/méthodes et peut les enrichir.

```
public class Etudiant extends Personne {
    private String filiere;

    public Etudiant(String nom, int age, String filiere) {
        super(nom, age); // appelle le constructeur de Personne
        this.filiere = filiere;
    }

    @Override
    public void direBonjour() {
        super.direBonjour(); // appelle la version de Personne
        System.out.println("Je suis en " + filiere);
    }
}
```

- **super(...)** : appelle le constructeur de la classe parente.
- **super.methode()** : appelle la méthode de la classe parente.
- **@Override** : indique qu'on redéfinit une méthode héritée (le compilateur vérifie la correspondance).

3. Interfaces

Une interface définit un **contrat** : une liste de méthodes que les classes qui l'implémentent doivent fournir.

```
public interface Mangeable {
    void manger();
}

public class Personne implements Mangeable {
    @Override
    public void manger() {
        System.out.println("Je mange");
    }
}
```

- Une classe peut implémenter **plusieurs interfaces** (implements A, B), mais hériter d'**une seule classe** (extends).
- Depuis Java 8, une interface peut avoir des méthodes avec corps : **default** et **static**.

```
public interface Mangeable {
    void manger();

    default void grignoter() {
        System.out.println("Je grignote");
    }
}
```

4. Polymorphisme

Le polymorphisme permet de manipuler des objets de types différents via une référence commune (classe parente ou interface).

```
Personne p = new Etudiant("Ainhua", 21, "Informatique");
p.direBonjour(); // exécute la version d'Etudiant, pas celle de Personne
```

C'est le **type réel de l'objet** qui détermine quelle méthode est appelée à l'exécution (liaison dynamique), pas le type de la référence.

```
List<Personne> groupe = new ArrayList<>();
groupe.add(new Personne("Marie", 30));
groupe.add(new Etudiant("Ainhua", 21, "Info"));

for (Personne p : groupe) {
    p.direBonjour(); // chaque objet exécute SA propre version
}
```

5. Classes abstraites

Une classe abstraite ne peut pas être instanciée directement ; elle sert de base commune avec des méthodes parfois non implémentées.

```
public abstract class Forme {
    public abstract double aire(); // pas de corps, à implémenter par les enfants

    public void afficherAire() {
        System.out.println("Aire : " + aire());
    }
}

public class Cercle extends Forme {
    private double rayon;

    public Cercle(double rayon) { this.rayon = rayon; }

    @Override
    public double aire() {
        return Math.PI * rayon * rayon;
    }
}
```

Classe abstraite vs interface

	Classe abstraite	Interface
Héritage	une seule (extends)	plusieurs (implements)
Attributs	oui (avec état)	non (sauf static final)
Constructeur	oui	non
Usage typique	base commune + logique partagée	contrat de comportement

JAVA — Thème 3

Collections & génériques

1. Les génériques

Les génériques permettent d'écrire du code qui fonctionne avec différents types tout en gardant la sécurité du typage à la compilation, sans cast manuel.

```
public class Boite<T> {  
    private T contenu;  
  
    public void mettre(T contenu) { this.contenu = contenu; }  
    public T sortir() { return contenu; }  
}
```

```
Boite<String> b = new Boite<>();  
b.mettre("Bonjour");  
String s = b.sortir(); // pas de cast nécessaire
```

- **T** est un paramètre de type (convention : T, E, K, V...).
- Le compilateur vérifie la cohérence des types à la compilation, évite les `ClassCastException` à l'exécution.

2. Les principales collections

List — ordonnée, doublons autorisés

```
List<String> noms = new ArrayList<>();  
noms.add("Ainhua");  
noms.add("Marie");  
noms.get(0); // "Ainhua"  
noms.remove("Marie");
```

- **ArrayList** : accès rapide par index, insertion/suppression lente au milieu.
- **LinkedList** : insertion/suppression rapide, accès par index plus lent.

Set — pas de doublons

```
Set<String> villes = new HashSet<>();  
villes.add("Brest");  
villes.add("Brest"); // ignoré, déjà présent  
// HashSet : pas d'ordre garanti  
// TreeSet : trié automatiquement  
// LinkedHashSet : garde l'ordre d'insertion
```

Map — paires clé/valeur

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Ainhua", 21);
ages.put("Marie", 30);
ages.get("Ainhua");           // 21
ages.getOrDefault("Tom", 0); // 0 si absent

for (Map.Entry<String, Integer> entry : ages.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

Interface	Implémentation courante	Particularité
List	ArrayList, LinkedList	Ordonnée, doublons autorisés
Set	HashSet, TreeSet, LinkedHashSet	Pas de doublons
Map	HashMap, TreeMap, LinkedHashMap	Clé → Valeur, clés uniques

3. Tri et comparaison

```
List<Personne> liste = new ArrayList<>();
// Tri avec Comparator (lambda)
liste.sort((p1, p2) -> p1.getAge() - p2.getAge());

// Ou via Comparator.comparing
liste.sort(Comparator.comparing(Personne::getNom));
```

JAVA — Thème 4

Streams & expressions lambda

1. Les expressions lambda

Une lambda est une fonction anonyme, utilisée pour passer un comportement en paramètre (souvent avec une interface fonctionnelle à une seule méthode).

```
// Syntaxe : (paramètres) -> expression
Runnable r = () -> System.out.println("Bonjour");

Comparator<Integer> comp = (a, b) -> a - b;

// Interface fonctionnelle personnalisée
interface Operation {
    int appliquer(int a, int b);
}
Operation addition = (a, b) -> a + b;
```

- **Interface fonctionnelle** : interface avec une seule méthode abstraite (annotée souvent @FunctionalInterface).
- Interfaces standards : `Function<T,R>`, `Predicate<T>`, `Consumer<T>`, `Supplier<T>`.

2. Les Streams

Un Stream permet de traiter une collection de façon déclarative : on enchaîne des opérations sans boucle explicite.

```
List<String> noms = List.of("Ainhua", "Marie", "Tom", "Alice");

List<String> resultat = noms.stream()
    .filter(n -> n.length() > 3) // garde les noms de plus de 3 lettres
    .map(String::toUpperCase)    // met en majuscules
    .sorted()                    // trie
    .collect(Collectors.toList()); // récupère le résultat

// resultat = [ALICE, AINHOA, MARIE]
```

Opérations courantes

Méthode	Rôle
<code>filter(predicate)</code>	garde les éléments qui valident une condition
<code>map(function)</code>	transforme chaque élément
<code>sorted()</code>	trie les éléments
<code>forEach(consumer)</code>	exécute une action sur chaque élément

<code>collect(...)</code>	récupère le résultat (liste, set, map...)
<code>reduce(...)</code>	agrège tous les éléments en une seule valeur
<code>count()</code>	compte les éléments

3. Exemple combiné

```
List<Personne> personnes = List.of(
    new Personne("Ainhoa", 21),
    new Personne("Marie", 30),
    new Personne("Tom", 17)
);

double moyenneAge = personnes.stream()
    .filter(p -> p.getAge() >= 18)
    .mapToInt(Personne::getAge)
    .average()
    .orElse(0);
```

À *retenir* : un Stream ne modifie pas la collection d'origine, il produit un nouveau résultat. Un Stream ne se consomme qu'une seule fois.

JAVA — Thème 5

Design Patterns courants

Les design patterns sont des solutions standard à des problèmes de conception récurrents. Voici les plus demandés en entretien/projet.

1. Singleton

Garantit qu'une classe n'a qu'une seule instance, accessible globalement.

```
public class Configuration {
    private static Configuration instance;

    private Configuration() {} // constructeur privé

    public static Configuration getInstance() {
        if (instance == null) {
            instance = new Configuration();
        }
        return instance;
    }
}
```

2. Factory (Fabrique)

Délègue la création d'objets à une méthode dédiée, pour ne pas exposer la logique d'instanciation au code appelant.

```
public interface Forme { void dessiner(); }
public class Cercle implements Forme {
    public void dessiner() { System.out.println("Cercle"); }
}
public class Carre implements Forme {
    public void dessiner() { System.out.println("Carré"); }
}

public class FormeFactory {
    public static Forme creer(String type) {
        return switch (type) {
            case "cercle" -> new Cercle();
            case "carre" -> new Carre();
            default -> throw new IllegalArgumentException("Type inconnu");
        };
    }
}
```

3. Builder

Construit un objet complexe étape par étape, utile quand un constructeur aurait trop de paramètres.

```
public class Pizza {
    private final String taille;
    private final boolean fromage;

    public static class Builder {
        private String taille = "M";
        private boolean fromage = false;

        public Builder taille(String t) { this.taille = t; return this; }
        public Builder fromage(boolean f) { this.fromage = f; return this; }
        public Pizza build() { return new Pizza(this); }
    }

    private Pizza(Builder b) {
        this.taille = b.taille;
        this.fromage = b.fromage;
    }
}

Pizza p = new Pizza.Builder().taille("L").fromage(true).build();
```

4. Observer

Permet à un objet (sujet) de notifier automatiquement une liste d'observateurs lorsqu'un événement se produit.

```
public interface Observateur { void notifier(String evenement); }

public class Sujet {
    private List<Observateur> observateurs = new ArrayList<>();

    public void abonner(Observateur o) { observateurs.add(o); }

    public void declencher(String evenement) {
        for (Observateur o : observateurs) {
            o.notifier(evenement);
        }
    }
}
```

5. Strategy

Permet de changer dynamiquement un algorithme (comportement) sans modifier la classe qui l'utilise — souvent implémenté avec une interface fonctionnelle.

```
public interface StrategiePaiement { void payer(double montant); }

public class PaiementCarte implements StrategiePaiement {
    public void payer(double montant) { System.out.println("Carte : " + montant); }
}

public class Commande {
    private StrategiePaiement strategie;
    public Commande(StrategiePaiement s) { this.strategie = s; }
    public void payer(double montant) { strategie.payer(montant); }
}
```

JAVA — Thème 6

Tests avec JUnit 5 & accès aux bases de données avec JDBC

1. JUnit 5 — les bases

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculTest {

    @Test
    void testAddition() {
        Calcul calcul = new Calcul();
        assertEquals(5, calcul.additionner(2, 3));
    }

    @Test
    void testDivisionParZero() {
        Calcul calcul = new Calcul();
        assertThrows(ArithmeticException.class, () -> calcul.diviser(10, 0));
    }
}
```

Annotation / Méthode	Rôle
@Test	marque une méthode comme test
@BeforeEach	exécuté avant chaque test (setup)
@AfterEach	exécuté après chaque test (nettoyage)
@BeforeAll / @AfterAll	exécuté une seule fois (static)
assertEquals(a, b)	vérifie que a == b
assertTrue / assertFalse	vérifie une condition booléenne
assertThrows(...)	vérifie qu'une exception est levée

2. Cycle de vie d'un test

```
class ExempleTest {

    @BeforeEach
    void setup() {
        System.out.println("Avant chaque test");
    }

    @Test
    void test1() { /* ... */ }

    @AfterEach
    void cleanup() {
        System.out.println("Après chaque test");
    }
}
```

3. JDBC — connexion à une base de données

JDBC (Java Database Connectivity) permet à une application Java de communiquer avec une base de données relationnelle (MySQL, PostgreSQL...).

```
import java.sql.*;

String url = "jdbc:mysql://localhost:3306/madb";
String user = "root";
String password = "motdepasse";

try (Connection conn = DriverManager.getConnection(url, user, password)) {

    String sql = "SELECT nom, age FROM personnes WHERE age > ?";
    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, 18);

        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                System.out.println(rs.getString("nom") + " - " + rs.getInt("age"));
            }
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

- **try-with-resources** : ferme automatiquement Connection/Statement/ResultSet.
- **PreparedStatement** : à privilégier sur Statement, protège des injections SQL.
- Le ? dans la requête est rempli ensuite via setInt, setString, etc.

4. Insertion / mise à jour

```
String sql = "INSERT INTO personnes (nom, age) VALUES (?, ?)";
try (PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, "Ainhua");
    stmt.setInt(2, 21);
    stmt.executeUpdate(); // pour INSERT/UPDATE/DELETE
}
```

Fin du programme de révision — Thèmes 1 à 6.